AI-Based Chess Game

Minimax Algorithm + Alpha-Beta Pruning
Practical and presentation-ready explanation

1. Project Overview

This project develops a console-based AI Chess game in Python. A human player controls White and the AI controls Black. The AI uses Minimax with Alpha-Beta Pruning to search possible future moves and select a move that gives Black the best evaluated outcome.

Core technologies: Python, python-chess, game-tree search, heuristic evaluation, Minimax and Alpha-Beta Pruning.

2. AI Approaches Used

Approach	Purpose
Game Theory / Zero-Sum Game	Models chess as a competition where an advantage for one player is a disadvantage for the other.
State Representation	Uses python-chess to represent pieces, turns, legal moves and game status.
Heuristic Evaluation	Converts a chess position into a numerical score.
Minimax	Searches future moves assuming both players make the best decisions.
Alpha-Beta Pruning	Skips branches that cannot affect the final Minimax decision.
Depth-Limited Search	Searches only a fixed number of levels instead of the entire game.

3. State Representation with python-chess

The program uses the python-chess library instead of implementing every chess rule manually. **chess.Board()** creates and maintains the game state.

- Stores the position of every piece.
- Tracks whose turn it is.
- Generates legal moves.
- Validates moves such as e2e4.
- Handles check, checkmate and stalemate.
- Allows moves to be made with push() and undone with pop().

4. Zero-Sum Evaluation

The evaluation is measured from White's perspective. Positive values mean White is better; negative values mean Black is better; zero means the position is approximately equal.

Score	Meaning
+10	Strong White advantage
+1	Small White advantage
0	Approximately equal
-1	Small Black advantage
-10	Strong Black advantage

5. Piece Values

Piece	Value	Use
Pawn	1	Basic material unit
Knight	3	Minor piece
Bishop	3	Minor piece
Rook	5	Major piece
Queen	9	Most valuable non-king piece
King	100	Very large practical value

6. Heuristic Board Evaluation

The AI cannot search a complete chess game to the end. Instead, it stops at a selected depth and asks how good the resulting position is. This is the heuristic evaluation.

- **Material:** remaining pieces multiplied by their values.
- **Center control:** a small bonus for pieces on d4, e4, d5 and e5.
- **Mobility:** a small bonus for having more legal moves.
- **Terminal positions:** checkmate receives a very large score and draws receive 0.

7. Minimax Algorithm

Minimax is the main decision-making algorithm. It builds a game tree by simulating possible moves. It assumes both players make optimal decisions.

White = Maximizing player: tries to make the evaluation as high as possible.

Black = Minimizing player: the AI tries to make the evaluation as low as possible.

Example: if Black can choose outcomes of -2, +1 and -5, it chooses -5 because Black is the minimizing player.

8. Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization of Minimax. It does not change the final Minimax answer. It stops exploring a branch once that branch can no longer improve the decision.

Alpha (α): best score the maximizing player can guarantee so far.

Beta (β): best score the minimizing player can guarantee so far.

When **beta <= alpha**, the remaining branch can be pruned.

```
if beta <= alpha:
    break
```

9. Why Alpha-Beta Pruning Matters

Chess has an enormous game tree. Searching every possible game is impractical. Alpha-Beta Pruning reduces unnecessary calculations while preserving the result that ordinary Minimax would produce.

10. find_best_move()

This is the AI's top-level decision function. It tests every legal Black move, temporarily plays each one, asks Minimax how White could respond, restores the board, and keeps the move with the lowest score.

- Start with the best score as positive infinity because Black wants a low score.
- Generate each legal Black move.
- Temporarily push the move.
- Run Minimax for the remaining depth.

- Pop the move to restore the board.
- Keep the move with the lowest score.
- Return the selected move.

11. Depth-Limited Search

The depth parameter controls how far ahead the AI searches. A small depth is fast but less intelligent; increasing depth usually improves lookahead but greatly increases computation.

Depth	Meaning
1	Very shallow and fast
2	Good for a simple practical demonstration
3	More lookahead and more computation
Higher	Rapidly growing search tree

12. Game Loop: play_game()

The play_game() function connects the search algorithm to an actual game. It repeatedly checks whose turn it is.

Human / White turn:

- Read a UCI move such as e2e4.
- Convert it using chess.Move.from_uci().
- Check whether the move is legal.
- Push the move onto the board.

AI / Black turn:

- Call find_best_move().
- Minimax searches future positions.
- Alpha-Beta Pruning eliminates unnecessary branches.
- The best Black move is returned.
- The AI permanently pushes the selected move.

13. Execution Example: User Enters e2e4

Step 1 — Input

The user enters e2e4.

Step 2 — Validation

python-chess converts the input into a move and checks whether it is legal.

Step 3 — State Update

board.push(move) moves the pawn and changes the turn to Black.

Step 4 — AI Search

Black calls find_best_move(board, depth=2).

Step 5 — Candidate Moves

The AI temporarily tries every legal Black move.

Step 6 — Minimax

For each candidate, Minimax simulates White responses.

Step 7 — Alpha-Beta

Branches that cannot affect the decision are stopped early.

Step 8 — Selection

Black chooses the move with the lowest evaluation.

Step 9 — Real Move

The selected move is pushed onto the actual board.

Step 10 — Next Turn

The program returns to White.

14. Important Python Operations

Operation	Meaning
board.push(move)	Makes a move.
board.pop()	Undoes the last move.
board.legal_moves	Generates legal moves.
board.turn	Shows whose turn it is.
board.is_game_over()	Checks whether the game ended.
board.is_checkmate()	Checks for checkmate.
chess.Move.from_uci()	Converts text such as e2e4 into a move object.

15. Overall Architecture

Human Move → **Board Update** → **AI Turn** → **find_best_move()** → **Minimax** → **Alpha-Beta Pruning** → **Heuristic Evaluation** → **Best Move** → **AI Move**

16. What Type of AI Is This?

This is a **search-based AI**, not a machine-learning model. It uses adversarial search and a heuristic evaluation function to make decisions. The project is a practical example of Game Playing AI and Heuristic Search.

17. Limitations

- The search depth is small compared with professional chess engines.
- The heuristic is simple and mainly considers material, center control and mobility.
- It does not learn from a training dataset.
- It does not use opening books or endgame tablebases.
- Professional engines such as Stockfish use much more advanced search and evaluation.

18. Why This Is Suitable for an AISC Practical

The project clearly demonstrates adversarial search, recursion, heuristic evaluation, state-space exploration and Alpha-Beta optimization. The result is visible because the AI actually generates chess moves.

- Demonstrates Minimax.
- Demonstrates Alpha-Beta Pruning.
- Demonstrates heuristic search.
- Demonstrates recursive decision-making.
- Produces an observable AI decision.

19. Viva Questions and Answers

Q: What is Minimax?

A: A two-player game decision algorithm that assumes both players make optimal decisions.

Q: Why is Black the minimizing player?

A: The evaluation is from White's perspective, so lower scores are better for Black.

Q: What is Alpha-Beta Pruning?

A: An optimization that skips branches that cannot change the Minimax result.

Q: What is alpha?

A: The best value currently guaranteed to the maximizing player.

Q: What is beta?

A: The best value currently guaranteed to the minimizing player.

Q: Why use a heuristic?

A: Because searching an entire chess game is computationally impractical.

Q: Why use depth?

A: To limit computation to a manageable number of future levels.

Q: Why use python-chess?

A: It provides reliable chess rules, legal move generation and board management.

Q: Is this machine learning?

A: No. It is a search-based AI using Minimax and a heuristic.

Q: Where is pruning performed?

A: At the beta <= alpha condition inside the Minimax function.

20. Final Presentation Summary

The project represents chess as a state-space search problem. The AI generates legal moves using python-chess, searches future positions with Minimax, uses Alpha-Beta Pruning to remove unnecessary branches, evaluates positions using a heuristic, and selects the best move for Black. This demonstrates adversarial Artificial Intelligence and heuristic search in a practical and easy-to-demonstrate application.

One-line viva answer:
“My project uses Minimax with Alpha-Beta Pruning to search possible chess moves, evaluate future board positions using a heuristic, and select the best move for the AI playing Black.”